

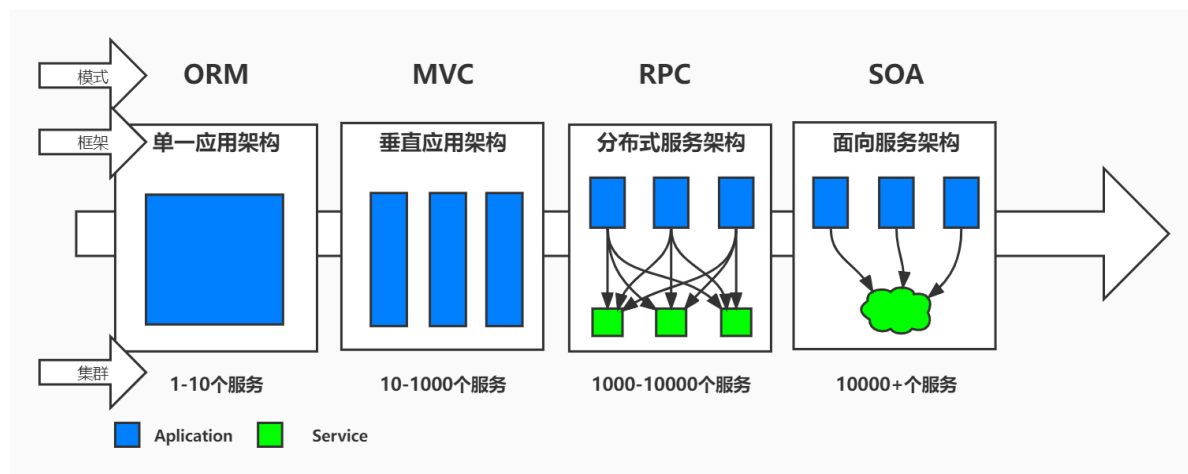
微服务SpringCloudAlibaba

授课目标

- 目标01-理解单体架构演进到微服务架构的过程
- 目标02-掌握微服务是什么，及微服务的优缺点
- 目标03-掌握SpringCloud是什么，及版本定义的规则与核心组件
- 目标04-掌握微服务案例环境搭建
- 目标05-掌握Nacos的安装与整合
- 目标06-掌握Nacos的集群搭建

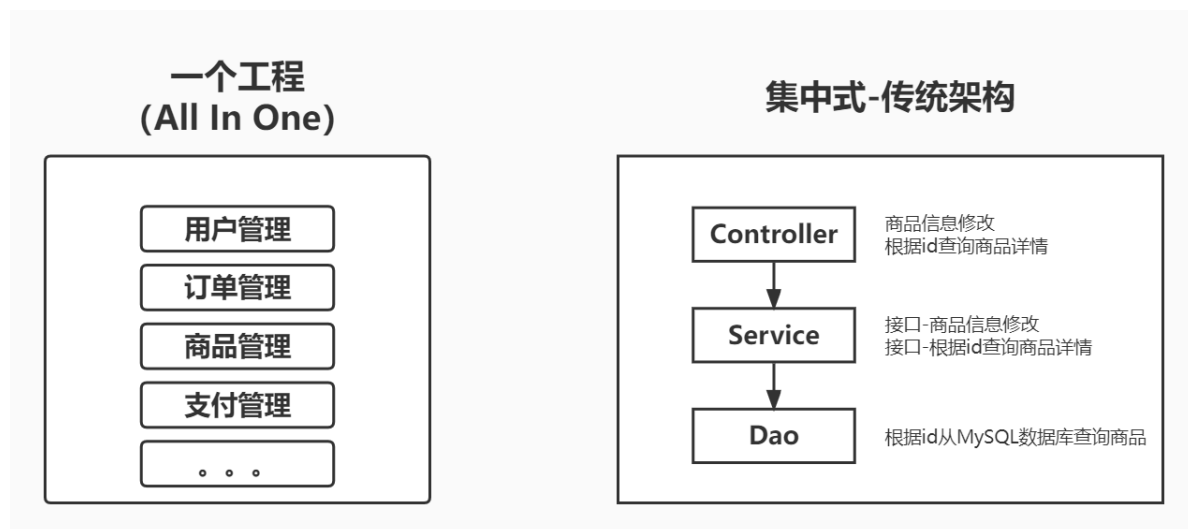
第1章 微服务

1.1 从单体架构到微服务架构的演进



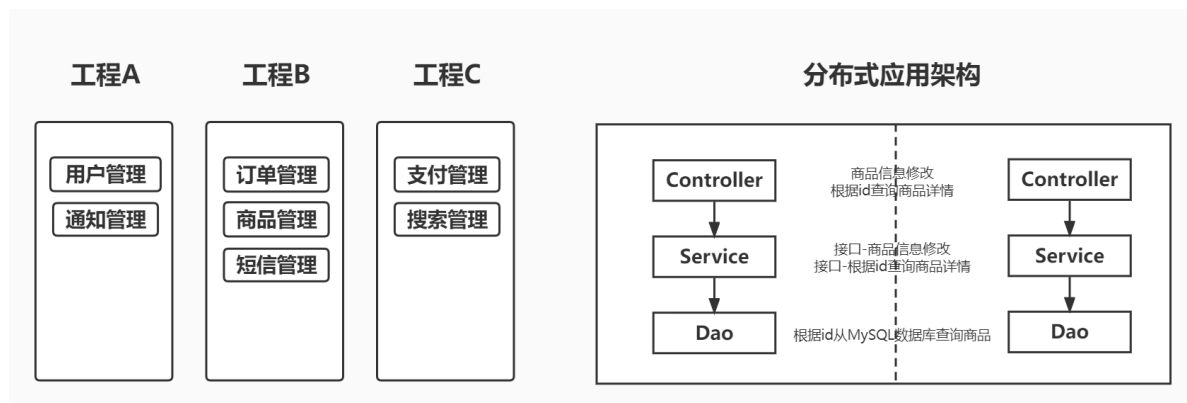
1.1.1 单一应用架构

当网站流量很小时，只需要一个应用，所有功能部署在一起。



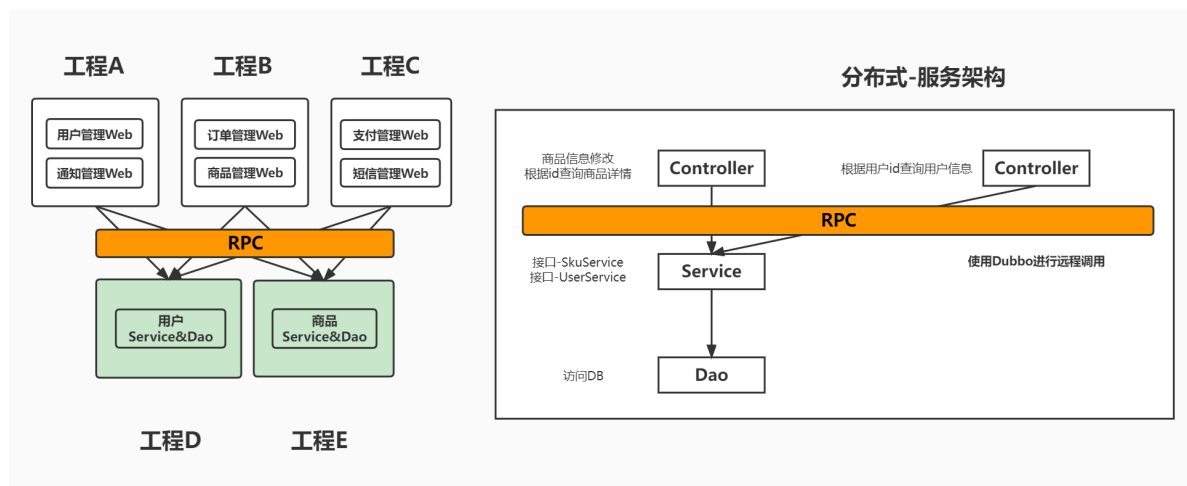
1.1.2 垂直应用架构

当访问量逐渐增大，单一应用增加机器带来的加速度越来越小，将应用拆成互不相干的几个应用，以提升效率。



1.1.3 分布式服务架构

当垂直应用越来越多，应用之间交互不可避免，将核心业务抽取出来，作为独立的服务，逐渐形成稳定的服务中心，使前端应用能更快速的响应多变的市场需求。此时，用于提高业务复用及整合的分布式服务框架(RPC)是关键。

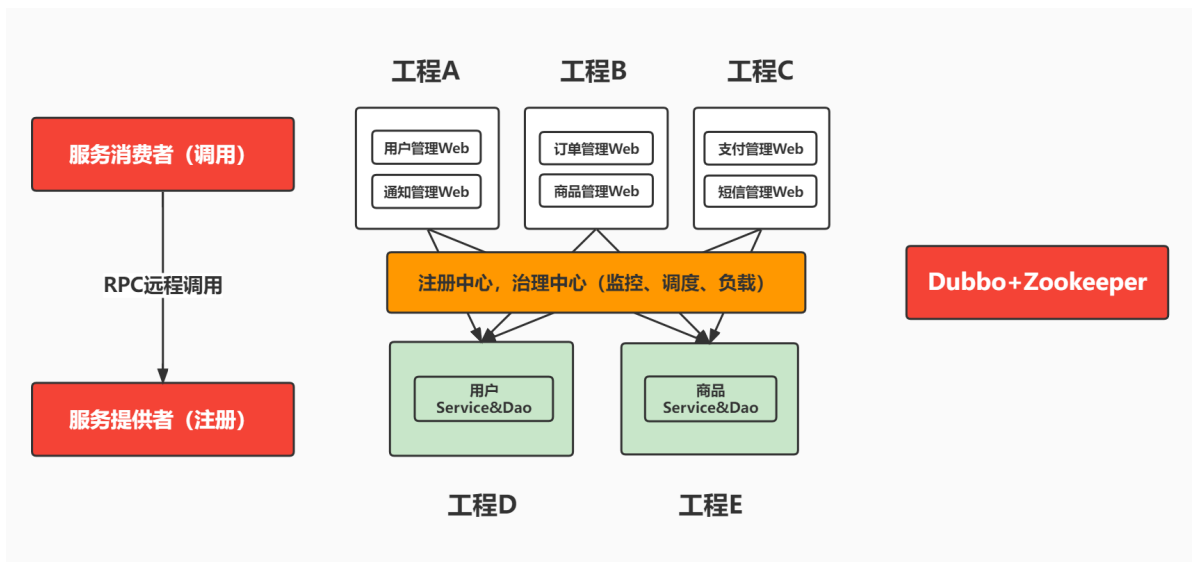


1.1.4 面向服务架构

1) 流动计算架构

当服务越来越多，容量的评估，小服务资源的浪费等问题逐渐显现，此时需增加一个调度中心基于访问压力实时管理集群容量，提高集群利用率。此时，用于提高机器利用率的资源调度和治理中心(SOA)是关键。

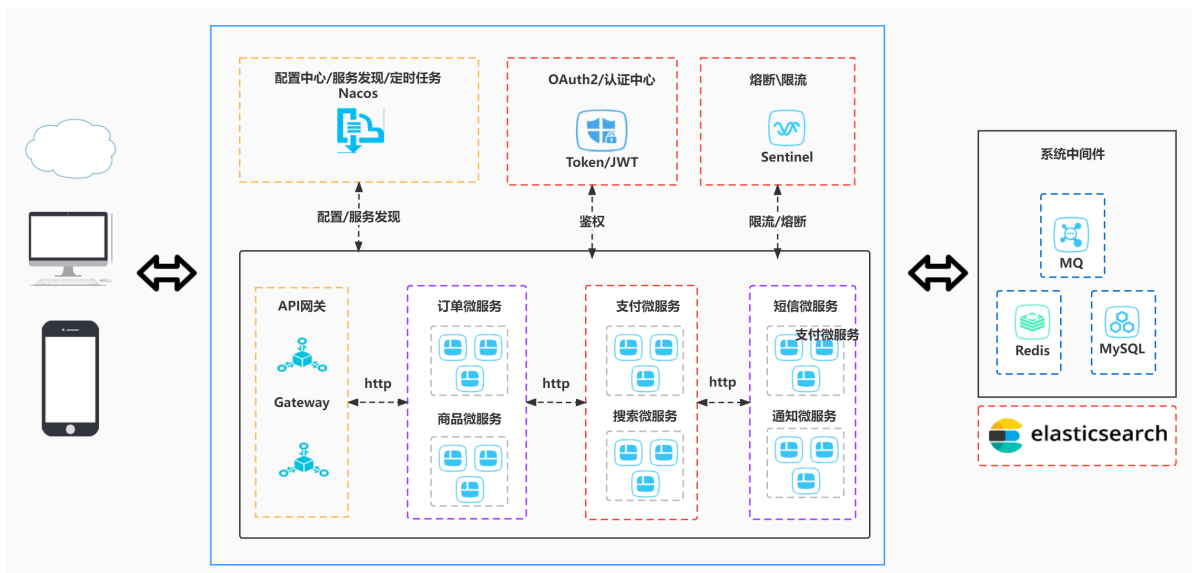
流动计算架构的最佳实践阿里的Dubbo。



2) 微服务架构

微服务架构与流动计算架构很相似，除了具备流动计算架构优势外，微服务架构中的微服务可以独立部署，独立发展。且微服务的开发不会限制于任何技术栈。

微服务架构的最佳实践是SpringCloud。



1.2 什么是微服务？

关于什么是微服务，Martin Fowler 的Paper有如下的定义：

The term "Microservice Architecture" has sprung up over the last few years to describe a particular way of designing software applications as suites of independently deployable services. **While there is no precise definition of this architectural style**, there are certain common characteristics around organization around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data.

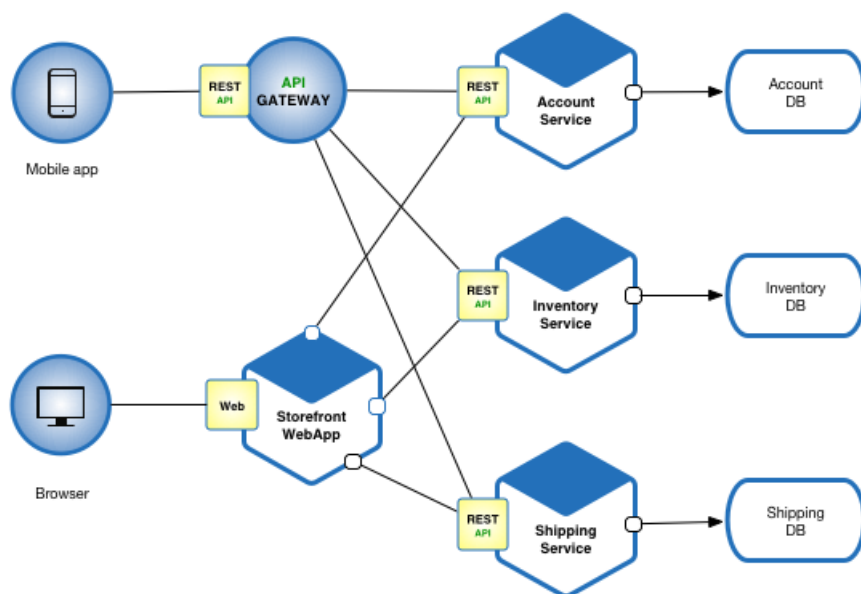
In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment

machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

这段话提到几个重要的点：**微服务架构的风格、一套小服务、独立进程、轻量级通信协、可独立部署、多语言&不同储技术**。可以这么理解：

- 微服务架构是一种架构风格
- 微服务把大的应用拆分为一组独立的小型服务
- 微服务每个服务运行在自己的进程内，可独立部署和独立升级
- 微服务的服务之间使用轻量级HTTP通信，一般使用JSON交换数据
- 微服务打破了语言的壁垒，可以使用不同语言、不同的存储技术

这个定义对微服务做了一个比较具象化较为易于理解的描述，通常来说我们看到的为服务架构如下图所示：**符合了基本特征，便可以称之为微服务了！**

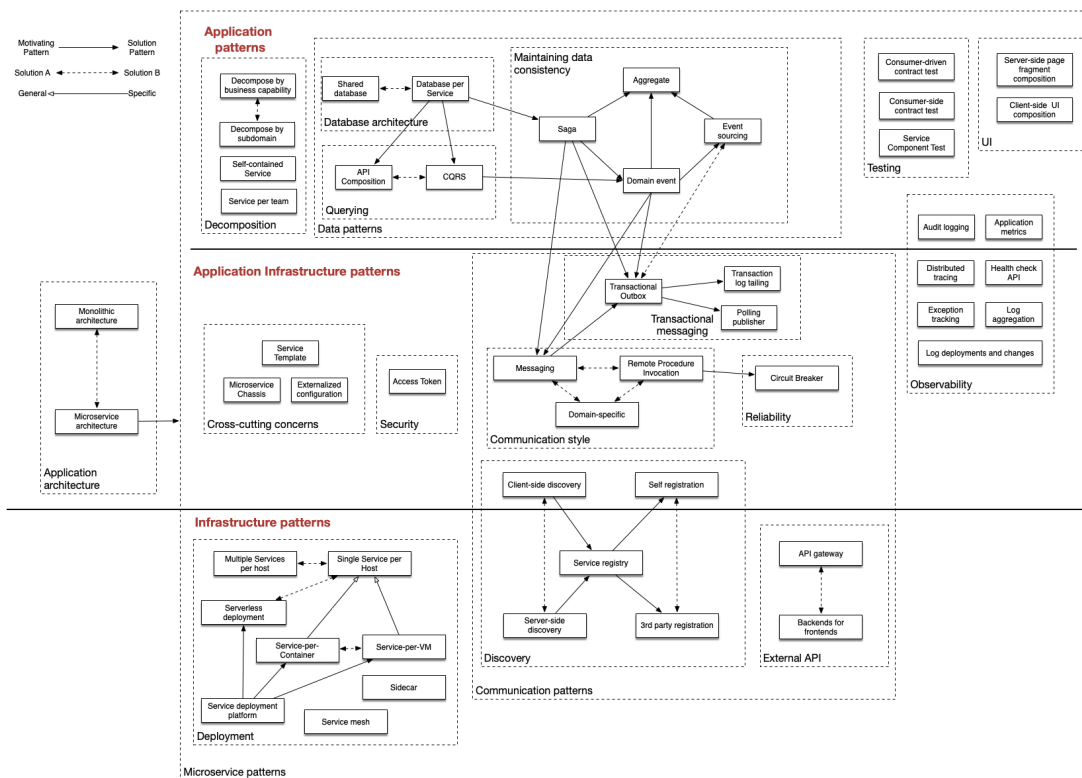


注：图片摘自<https://martinfowler.com>网站

在实际生产环境中，微服务的架构要考虑的问题远比上面的示意图复杂的多，主要包括如下问题：

1. 通过服务实现组件化
2. 根据业务组织系统
3. 简单高效的通信协议
4. 自动化的基础设施
5. 面向失败的设计
6. 具备进化能力的设计

微服务 不是一个被设计出来的架构，而应该是不断尝试中总结出的一套适合在互联网行业使用的架构模式。



Copyright © 2020, Chris Richardson Consulting, Inc. All rights reserved.

Learn-Build-Assess Microservices <http://adopt.microservices.io>

注：图片摘自<https://martinfowler.com>网站

参考文章：

1. MartinFowler关于微服务完整的定义<https://martinfowler.com/articles/microservices.htm>
2. 微服务架构模式：<https://microservices.io/patterns/cn/index.html>

1.2.1 微服务并不是银弹！

微服务并不能一劳永逸的解决了所有的问题，相反的，如果不能正确的使用微服务，则有可能被微服务弊端拖入泥潭：

- **分布式的代价：**原本在单体应用中，很多简单的问题都会在分布式环境下被几何级的放大。例如：分布式事务、分布式锁、远程调用等等。不能只考虑如何实现，相关场景的异常处理也是必须要考虑到的问题。
- **协同代价：**如果你经历过一个项目上线需要发布十几个应用，而这些应用又分别由多个团队在维护。你就能深刻的体会到了。
- **服务拆分需要很强的设计功力：**微服务的各种优势，其中一个重要的基础是对服务领域的正确切分。如果使用了不合适的切分粒度，或者是错误的切分方法，都会让服务不能很好的实现高内聚低耦合的要求。

1.2.2 什么是分布式？

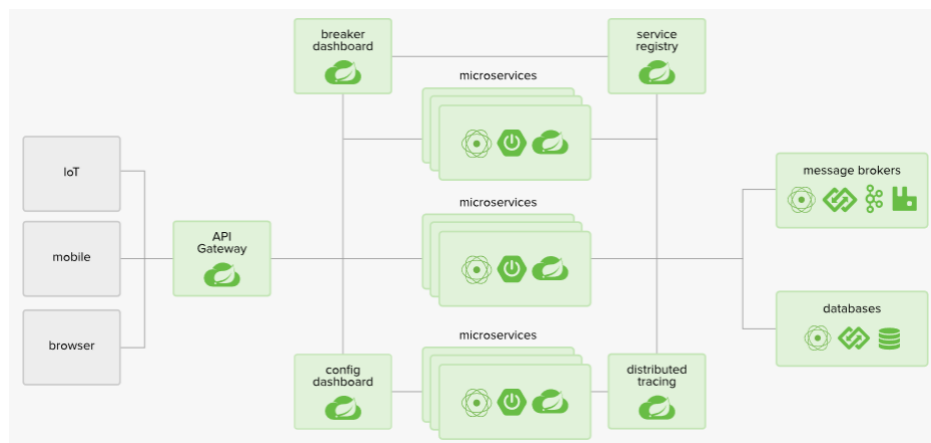
分布式系统是多个处理机通过通信线路互联而构成的松散耦合的系统。从系统中某台处理机来看，其余的处理机和相应的资源都是远程的，只有它自己的资源才是本地的。至今，对分布式系统的定义尚未形成统一的见解。一般认为，分布式系统应具有以下几个特征：

- **资源共享：**分布式系统可以共享硬件、软件或数据。
- **并行处理：**多台机器可以同时处理同一功能

- **公开透明**：节点可以访问系统中的其他节点并与之通信
- **错误检测**：可以更轻松地检测故障
- **支持扩展**：当扩充到其他计算机时，计算和处理能力可以按需进行扩展

1.3 什么是Spring Cloud?

SpringCloud 为开发人员提供了在分布式系统中快速构建一些常见模块的工具（例如：配置管理、服务发现、断路器、智能路由、微代理、控制总线、一次性令牌、全局锁、领导选举、分布式会话、集群状态）。Spring Cloud为每一个微服务节点提供一种开发样板，业务开发人员按照样板结构去开发即可。它们在任何分布式环境下都能很好地工作，包括开发人员自己的笔记本电脑、数据中心和云计算平台等。



注：图片摘自<https://spring.io>官方网站

官网：<https://spring.io/projects/spring-cloud>

Spring Cloud 是一系列框架【技术】的有序集合。它利用 Spring Boot 的开发便利性巧妙地简化了分布式系统基础设施的开发，如服务发现注册、配置中心、消息总线、负载均衡、断路器、数据监控等，都可以用 Spring Boot 的开发风格做到一键启动和部署。

Spring Cloud 并没有重复制造轮子，它只是将目前各家公司开发的比较成熟、经得起实际考验的服务框架组合起来，通过 Spring Boot 风格进行再封装屏蔽掉了复杂的配置和实现原理，最终给开发者提供了一套简单易懂、易部署和易维护的分布式系统开发工具包。

Spring Cloud 是什么?

- Spring Cloud 是微服务系统架构的一站式解决方案。**Spring Cloud 是一系列微服务节点的开发模板的合集。**

Spring Cloud 与 Spring Boot 是什么关系呢?

- SpringCloud里的基础组件几乎都是基于SpringBoot构建，SpringBoot是SpringCloud的基石，它为 Spring Cloud 提供了实现环境，也大大降低了搭建微服务平台的成本。
- 当然，我们也可以说SpringCloud 是**基于SpringBoot**的微服务系统架构的一站式解决方案

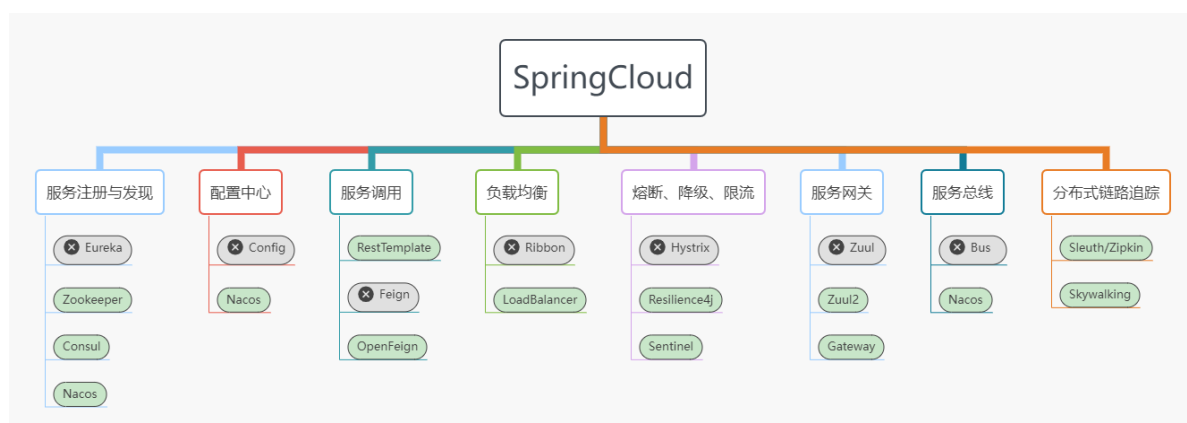
1.3.1 什么是Spring Cloud Alibaba?

Spring Cloud Alibaba 致力于提供微服务开发的一站式解决方案。此项目包含开发分布式应用服务的必需组件，方便开发者通过 Spring Cloud 编程模型轻松使用这些组件来开发分布式应用服务。

依托 Spring Cloud Alibaba，您只需要添加一些注解和少量配置，就可以将 Spring Cloud 应用接入阿里分布式应用解决方案，通过阿里中间件来迅速搭建分布式应用系统。

1.3.2 核心组件

SpringCloud的本质是在 Spring Boot 的基础上，增加了一堆微服务相关的规范，并对应用上下文（Application Context）进行了自动化配置。既然 Spring Cloud 是规范，那么就需要去实现，目前 Spring Cloud 规范已有 Spring官方，Spring Cloud Netflix，Spring Cloud Alibaba等实现。通过组件化的方式，Spring Cloud将这些实现整合到一起构成全家桶式的微服务技术栈。



1.3.3 版本定义

Documentation	
Each Spring project has its own; it explains in great details how you can use project features and what you can achieve with them.	
2021.0.5 CURRENT GA	Reference Doc.
2022.0.0-SNAPSHOT SNAPSHOT	Reference Doc.
2022.0.0-RC2 PRE	Reference Doc.
2021.0.6-SNAPSHOT SNAPSHOT	Reference Doc.
2020.0.7-SNAPSHOT SNAPSHOT	Reference Doc.
2020.0.6 GA	Reference Doc.
Hoxton.SR12 GA	Reference Doc.
Hoxton.BUILD-SNAPSHOT SNAPSHOT	Reference Doc.

大版本：

Spring Cloud 的版本号并不是我们通常见的数字版本号，而是一些很奇怪的单词。这些单词多数为英国伦敦地铁站的站名或地名。例如：Angel、Brixton、Camden、Dalston、Edgware、Greenwich、Hoxton、Ilford、Jubilee、Kilburn

从Ilford开始，加了一个日期前缀

小版本：

Spring Cloud 的一个大版本在不同的阶段会发布不同类型的小版本号。按照发行的顺序，一般会存在这些版本，但并不一定每个大版本都存在这些小版本。当然，这些小版本后一般还会添加上数字作为其内部的版本。

- PRE: preview, 预览版, 内测版。
- SNAPSHOT: 快照版。
- M 版: Milestone, 里程碑版。
- RC 版: Release Candidate, 发行候选版本。
- SR 版: Service Release, 服务发布版。
- GA: General Availability, 则表示这是当前广泛使用的版本。
- CURRENT: 表示官方当前推荐版本。

1.3.4 Spring Cloud与Spring Boot版本对应关系

Spring Cloud	Spring Boot
2022.0.x aka Kilburn	3.0.x
2021.0.x aka Jubilee	2.6.x, 2.7.x (Starting with 2021.0.3)
2020.0.x aka Ilford	2.4.x, 2.5.x (Starting with 2020.0.3)
Hoxton	2.2.x, 2.3.x (Starting with SR5)
Greenwich	2.1.x
Finchley	2.0.x
Edgware	1.5.x
Dalston	1.5.x

Dalston、Edgware、Finchley 和 Greenwich 都已达到生命周期终止状态，不再受支持。

1.4 版本说明

使用Spring Cloud Alibaba 时特别需要注意版本间的兼容关系，由于 Spring Boot 2.4+ 和以下版本之间变化较大，而国内企业SpringBoot版本任然停留在2.4以内，Alibaba为了兼顾存量用户和新用户的不同诉求，因此同时维护 2.2.x 和 2021.x 两个分支迭代。

2021.x 分支

Spring Cloud Alibaba Version	Spring Cloud Version	Spring Boot Version
2021.0.4.0*	Spring Cloud 2021.0.4	2.6.11
2021.0.1.0	Spring Cloud 2021.0.1	2.6.3
2021.1	Spring Cloud 2020.0.1	2.4.2

2.2.x 分支

Spring Cloud Alibaba Version	Spring Cloud Version	Spring Boot Version
2.2.9.RELEASE*	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.8.RELEASE	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.7.RELEASE	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.6.RELEASE	Spring Cloud Hoxton.SR9	2.3.2.RELEASE
2.2.1.RELEASE	Spring Cloud Hoxton.SR3	2.2.5.RELEASE
2.2.0.RELEASE	Spring Cloud Hoxton.RELEASE	2.2.X.RELEASE
2.1.4.RELEASE	Spring Cloud Greenwich.SR6	2.1.13.RELEASE
2.1.2.RELEASE	Spring Cloud Greenwich	2.1.X.RELEASE
2.0.4.RELEASE(停止维护, 建议升级)	Spring Cloud Finchley	2.0.X.RELEASE
1.5.1.RELEASE(停止维护, 建议升级)	Spring Cloud Edgware	1.5.X.RELEASE

官网: <https://github.com/alibaba/spring-cloud-alibaba/wiki>

版本说明: <https://github.com/alibaba/spring-cloud-alibaba/wiki/%E7%89%88%E6%9C%ACE8%AF%B4%E6%98%8E>

关于版本的选用, 只要在维护范畴内即可, 因此本课程采用的Spring Cloud, Spring Boot版本

- Spring Boot 2.4.2
- Spring Cloud 2020.0.1
- **Spring Cloud Alibaba 2021.1**

1.5 案例环境搭建

接下来模拟一个微服务调用的场景, 有两个微服务, 一个订单微服务, 一个支付微服务, 订单微服务调用支付微服务, 完成支付功能。

- 支付微服务工程
- 订单微服务工程

1.5.1 公共工程

使用maven创建公共工程spring_cloud_common工程下, 统一管理案例使用的实体类和工具类。

pom.xml

```

2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.hero</groupId>
8     <artifactId>spring-cloud-common</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>8</maven.compiler.source>
13         <maven.compiler.target>8</maven.compiler.target>
14     </properties>
15
16     <dependencies>
17         <dependency>
18             <groupId>org.projectlombok</groupId>
19             <artifactId>lombok</artifactId>
20             <version>1.18.20</version>
21         </dependency>
22     </dependencies>
23
24 </project>

```

Payment实体类：订单和支付工程都用到了

```

1 @Data
2 @NoArgsConstructor
3 @AllArgsConstructor
4 public class Payment {
5
6     /**
7      * 订单编号
8      */
9     private Integer id;
10    /**
11     * 支付状态
12     */
13    private String message;
14
15 }

```

1.5.2 支付微服务工程

(1) 创建支付微服务工程01_payment，导入依赖

- Spring Boot 2.4.2
- Spring Boot DevTools
- Lombok
- Spring Web

pom.xml配置文件如下

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>2.4.2</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11
12    <modelVersion>4.0.0</modelVersion>
13    <artifactId>01-payment</artifactId>
14
15    <properties>
16        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
17        <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
18        <java.version>1.8</java.version>
19    </properties>
20
21    <dependencies>
22
23        <dependency>
24            <groupId>org.springframework.boot</groupId>
25            <artifactId>spring-boot-starter-web</artifactId>
26        </dependency>
27        <dependency>
28            <groupId>org.springframework.boot</groupId>
29            <artifactId>spring-boot-devtools</artifactId>
30            <scope>runtime</scope>
31            <optional>true</optional>
32        </dependency>
33        <dependency>
34            <groupId>org.springframework.boot</groupId>
35            <artifactId>spring-boot-starter-test</artifactId>
36            <scope>test</scope>
37        </dependency>
38        <dependency>
39            <groupId>com.hero</groupId>
40            <artifactId>spring-cloud-common</artifactId>
41            <version>1.0-SNAPSHOT</version>
42        </dependency>
43    </dependencies>
44
45    <build>
46        <plugins>
47            <plugin>
48                <groupId>org.springframework.boot</groupId>
49                <artifactId>spring-boot-maven-plugin</artifactId>
50            </plugin>
51        </plugins>
52    </build>
53
```

```
54 | </project>
```

(2) 启动器

```
1 | @SpringBootApplication
2 | public class PaymentApplication {
3 |
4 |     public static void main(String[] args) {
5 |         SpringApplication.run(PaymentApplication.class, args);
6 |     }
7 | }
```

(3) 配置文件

```
1 | server.port: 8081
```

(4) PaymentController: 在PaymentController中模拟实现支付功能, 代码如下

```
1 | @RestController
2 | @RequestMapping("/payment")
3 | public class PaymentController {
4 |
5 |     @GetMapping("/{id}")
6 |     public ResponseEntity<Payment> payment(@PathVariable("id") Integer id) {
7 |         Payment payment = new Payment(id, "支付成功");
8 |         return ResponseEntity.ok(payment);
9 |     }
10 |
11 | }
```

1.5.3 订单微服务

(1) 创建支付微服务工程02_order, 导入依赖

- Spring Boot 2.4.2
- Spring Boot DevTools
- Lombok
- Spring Web

pom.xml代码如下

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5 | http://maven.apache.org/xsd/maven-4.0.0.xsd">
6 |     <parent>
7 |         <groupId>org.springframework.boot</groupId>
8 |         <artifactId>spring-boot-starter-parent</artifactId>
9 |         <version>2.4.2</version>
10 |        <relativePath/> <!-- lookup parent from repository -->
11 |     </parent>
```

```

11
12     <modelVersion>4.0.0</modelVersion>
13
14     <artifactId>01-order</artifactId>
15
16     <dependencies>
17         <dependency>
18             <groupId>org.springframework.boot</groupId>
19             <artifactId>spring-boot-starter-web</artifactId>
20         </dependency>
21         <dependency>
22             <groupId>org.springframework.boot</groupId>
23             <artifactId>spring-boot-devtools</artifactId>
24             <scope>runtime</scope>
25             <optional>true</optional>
26         </dependency>
27         <dependency>
28             <groupId>org.springframework.boot</groupId>
29             <artifactId>spring-boot-starter-test</artifactId>
30             <scope>test</scope>
31         </dependency>
32         <dependency>
33             <groupId>com.hero</groupId>
34             <artifactId>spring-cloud-common</artifactId>
35             <version>1.0-SNAPSHOT</version>
36         </dependency>
37     </dependencies>
38 </project>

```

(2) 启动器

```

1  @SpringBootApplication
2  public class OrderApplication {
3
4      public static void main(String[] args) {
5          SpringApplication.run(OrderApplication.class, args);
6      }
7
8      /**
9       * 注入RestTemplate模板对象，用来发送http请求
10     */
11     @Bean
12     public RestTemplate restTemplate() {
13         return new RestTemplate();
14     }
15 }

```

Spring提供了一个RestTemplate模板工具类，对基于Http的客户端进行了封装，并且实现了对象与json的序列化和反序列化，非常方便。RestTemplate并没有限定Http的客户端类型，而是进行了抽象，目前常用的3种都有支持：HttpClient、OkHttp、JDK原生的URLConnection（默认的）

(3) OrderController

```

1  @RestController
2  @RequestMapping("/order")
3  public class OrderController {
4
5      @Autowired
6      private RestTemplate restTemplate;
7
8      @GetMapping("/payment/{id}")
9      public ResponseEntity<Payment> getPaymentById(@PathVariable("id")
Integer id) {
10         //发送http请求的, http://localhost:8081地址写死, 去访问提供者行不行呢?
11         String url = "http://localhost:8081/payment/" + id;
12         Payment payment = restTemplate.getForObject(url, Payment.class);
13         return ResponseEntity.ok(payment);
14     }
15
16 }

```

1.5.4 小结

简单回顾一下，刚才我们写了什么：

- payment-service：对外提供了支付的接口
- order-service：通过RestTemplate访问 <http://localhost:8081/order/{id}> 接口，调用支付服务

这么做存在什么问题？

- 消费者直接连接的提供者
- 在order中，我们把url地址硬编码到了代码中，不方便后期维护
- order需要记忆payment-service的地址，如果出现变更，可能得不到通知，地址将失效
- order不清楚payment-service的状态，服务宕机也不知道
- payment-service只有1台服务，不具备高可用性
- 即便payment-service形成集群，order还需自己实现负载均衡

其实上面说的这些问题，总的来说就是分布式服务必然要面临的**服务治理**的问题：

- 服务管理
 - 如何自动注册和发现
 - 如何实现状态监管
 - 如何实现动态路由
- 服务如何实现负载均衡
- 服务如何解决容灾问题
- 服务如何实现统一配置

以上的问题，我们都将在SpringCloud中得到答案。

第2章 服务注册与发现Nacos

2.1 Nacos 概述

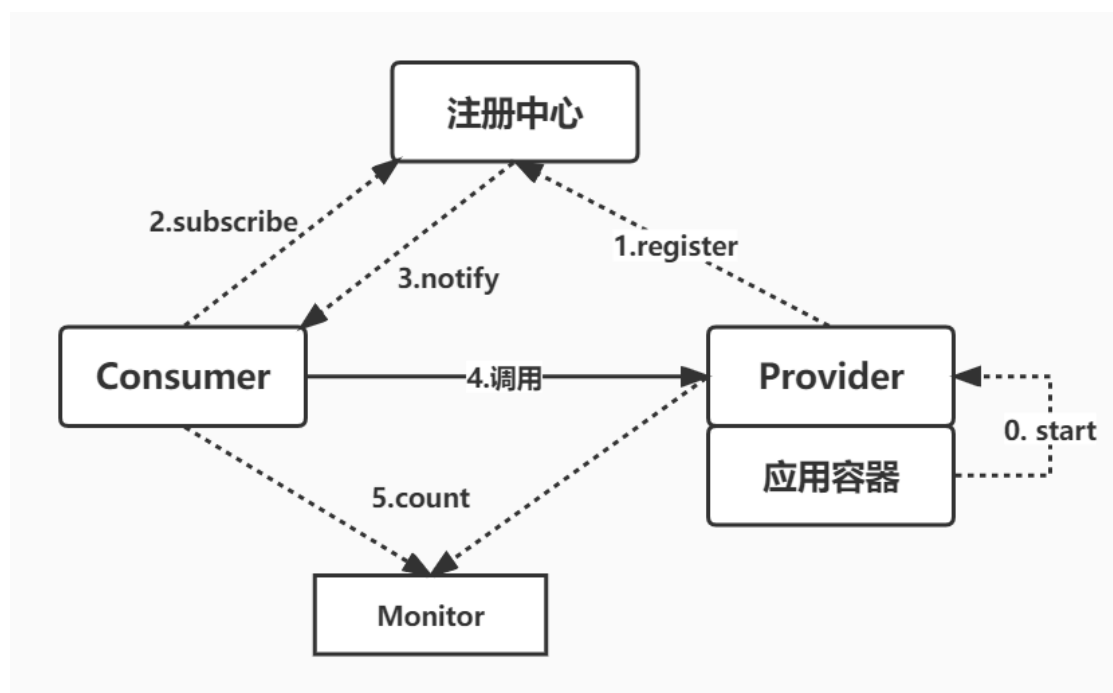
2.1.1 注册中心简介

前面的例子存在的问题：消费者直连提供者。这样做若提供者出现宕机，或消费者存在高并发情况，那么提供者就会出现各种问题。服务注册与发现就是这个问题的解决方案。所有提供者将自己提供服务的名称及自己主机详情（IP、端口、版本等）写入到另一个服务的列表中进行集中式管理，这个服务的职责就是管理所有服务的清单/列表，这种类型的服务一般称之为**服务注册中心**，而这个表称为**服务注册列表**。

所有消费者需要调用微服务时，其会从注册中心首先将服务注册表下载到本地，然后根据消费者本地设置好的负载均衡策略选择一个服务提供者进行调用。

可以充当 Spring Cloud 服务注册中心的服务器很多，如 Zookeeper、Eureka、Consul 等。Spring Cloud Alibaba 中使用的注册中心为 Nacos。

基本原理：



2.1.2 什么是Nacos?

Nacos 官网 <https://nacos.io/>

一个更易于构建云原生应用的动态服务发现、配置管理和服务管理平台。**直白了说：既是一个注册中心也是一个配置中心。**

服务 (Service) 是 Nacos 世界的一等公民。Nacos 支持几乎所有主流类型的“服务”的发现、配置和管理：

- [Kubernetes Service](#)
- [gRPC](#) & [Dubbo RPC Service](#)
- [Spring Cloud RESTful Service](#)

Nacos 的关键特性包括:

- 服务发现和服务健康监测
- 动态配置服务

- 动态 DNS 服务
- 服务及其元数据管理
- ...

2.2 Nacos 下载配置与安装

2.2.1 下载

▼ Assets 4

nacos-server-1.4.1.tar.gz	75.4 MB	Jan 15, 2021
nacos-server-1.4.1.zip	75.4 MB	Jan 15, 2021
Source code (zip)		Jan 14, 2021
Source code (tar.gz)		Jan 14, 2021

2.2.2 配置

这里下载 nacos-server-1.4.1.zip 这个打过包的 Server。解压到 D 盘根目录。

nacos-server-1.4.1 > nacos > conf

名称	修改日期	类型	大小
1.4.0-ipv6_support-update.sql	2020/12/24 20:15	SQL 文件	2 KB
application.properties	2021/1/15 9:43	PROPERTIES 文件	7 KB
application.properties.example	2020/12/24 20:15	EXAMPLE 文件	7 KB
cluster.conf.example	2020/12/15 15:29	EXAMPLE 文件	1 KB
nacos-logback.xml	2021/1/15 9:43	XML 文件	26 KB
nacos-mysql.sql	2020/12/24 20:15	SQL 文件	11 KB
schema.sql	2020/12/24 20:15	SQL 文件	9 KB

默认 Nacos 服务器的端口号为 8848，上下文路径为/nacos。一般都是采用默认值，但也可以修改。在 nacos/conf/application.properties 中：

```

17 #***** Spring Boot Related Configurations *****#
18 ### Default web context path:
19 server.servlet.contextPath=/nacos
20 ### Default web server port:
21 server.port=8848

```

2.2.3 Nacos 启动

(1) 启动

在 nacos/bin 目录中有启动命令文件。

: > nacos-server-1.3.1 > nacos > bin

名称	修改日期	类型	大小
shutdown.cmd	2020/5/14 10:03	Windows 命令脚本	1 KB
shutdown.sh	2020/7/3 18:25	Shell Script	1 KB
startup.cmd	2020/7/10 17:27	Windows 命令脚本	4 KB
startup.sh	2020/7/10 17:27	Shell Script	5 KB

其中 cmd 是 Windows 系统中的命令，sh 是 Linux 系统中的命令。

```

1 #命令启动
2 startup.cmd -m standalone

```

当看到如下日志时，表示启动成功。


```
C:\Windows\System32\cmd.exe - startup.cmd -m standalone

2022-12-07 12:29:03,956 INFO Bean 'org.springframework.security.access.expression.method.DefaultMethodSecurityExpression
d by all BeanPostProcessors (for example: not eligible for auto-proxying)

2022-12-07 12:29:04,069 INFO Bean 'methodSecurityMetadataSource' of type [org.springframework.security.access.method.D

2022-12-07 12:29:05,222 INFO Tomcat initialized with port(s): 8848 (http)

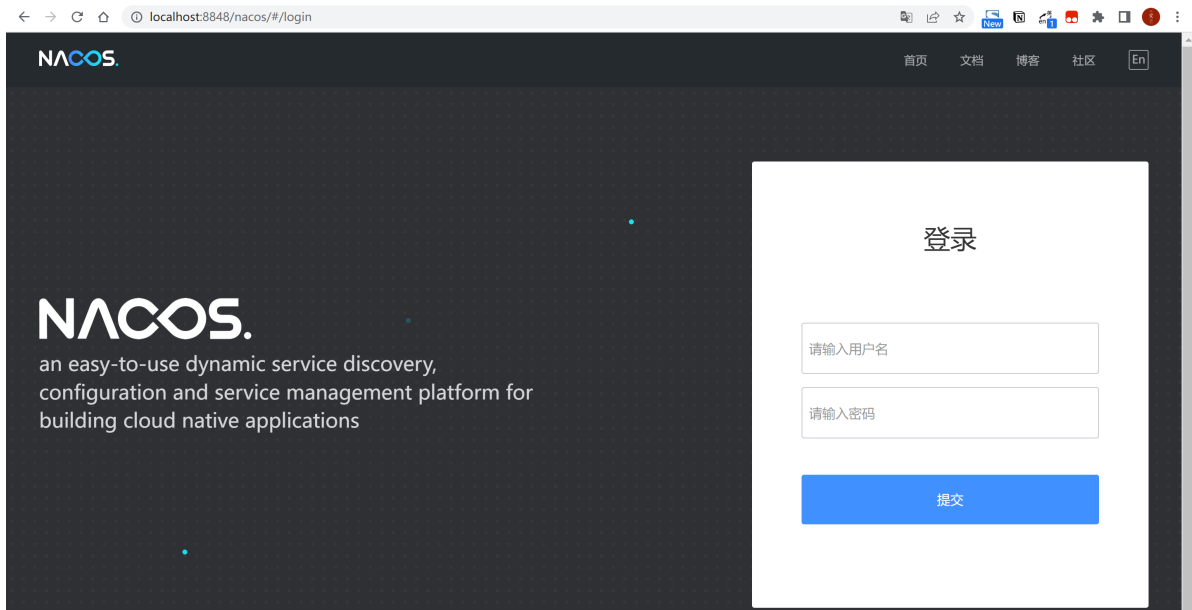
2022-12-07 12:29:05,866 INFO Root WebApplicationContext: initialization completed in 7250 ms

2022-12-07 12:29:11,558 INFO Initializing ExecutorService 'applicationTaskExecutor'

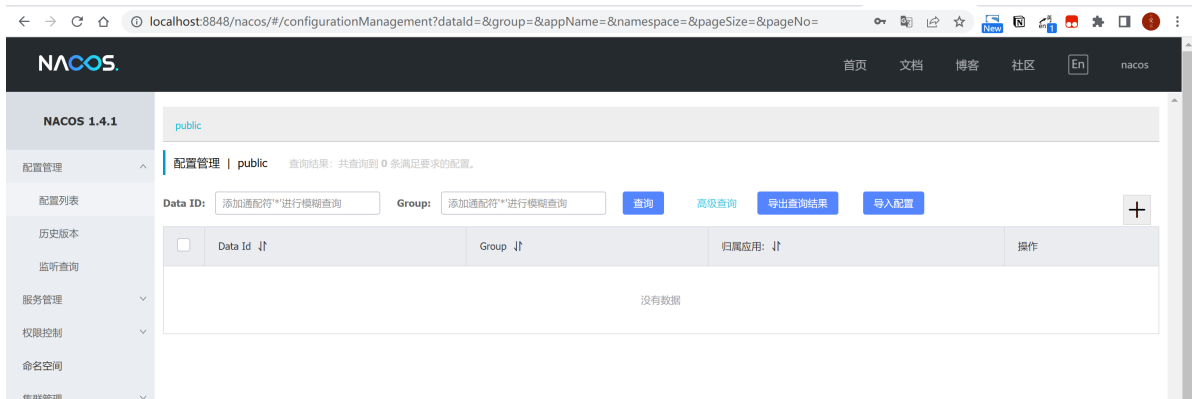
2022-12-07 12:29:11,715 INFO Adding welcome page: class path resource [static/index.html]
```

(2) 控制台

地址: <http://localhost:8848/nacos>



默认账号/密码为 nacos/nacos。输入账号/密码后即可看到如下界面，说明 nacos server 已经启动成功了。



2.3 整合Nacos

2.3.1 改造01-payment微服务

1) 复制工程

复制 01-payment工程，并重命名为 02-payment-nacos。

2) 修改 pom

在 pom 中添加了如下依赖：

- spring cloud 依赖
- spring cloud alibaba 依赖
- nacos-discovery 依赖

```
1  <properties>
2      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
3      <project.reporting.outputEncoding>UTF-
4      8</project.reporting.outputEncoding>
5      <java.version>1.8</java.version>
6      <spring-cloud.version>2020.0.1</spring-cloud.version>
7      <spring-cloud-alibaba.version>2021.1</spring-cloud-alibaba.version>
8  </properties>
9
10 <dependencies>
11     <!--nacos discovery依赖-->
12     <dependency>
13         <groupId>com.alibaba.cloud</groupId>
14         <artifactId>spring-cloud-starter-alibaba-nacos-
15         discovery</artifactId>
16     </dependency>
17
18     <dependency>
19         <groupId>org.springframework.boot</groupId>
20         <artifactId>spring-boot-starter-web</artifactId>
21     </dependency>
22
23     <dependency>
24         <groupId>org.springframework.boot</groupId>
25         <artifactId>spring-boot-devtools</artifactId>
26         <scope>runtime</scope>
27         <optional>true</optional>
28     </dependency>
29
30     <dependency>
31         <groupId>org.springframework.boot</groupId>
32         <artifactId>spring-boot-starter-test</artifactId>
33         <scope>test</scope>
34     </dependency>
35
36     <dependency>
37         <groupId>com.hero</groupId>
38         <artifactId>spring-cloud-common</artifactId>
39         <version>1.0-SNAPSHOT</version>
40     </dependency>
41 </dependencies>
42
43 <dependencyManagement>
44     <dependencies>
45         <dependency>
46             <groupId>org.springframework.cloud</groupId>
47             <artifactId>spring-cloud-dependencies</artifactId>
48             <version>${spring-cloud.version}</version>
```

```

43         <type>pom</type>
44         <scope>import</scope>
45     </dependency>
46     <dependency>
47         <groupId>com.alibaba.cloud</groupId>
48         <artifactId>spring-cloud-alibaba-dependencies</artifactId>
49         <version>${spring-cloud-alibaba.version}</version>
50         <type>pom</type>
51         <scope>import</scope>
52     </dependency>
53 </dependencies>
54 </dependencyManagement>
55
56 <build>
57     <plugins>
58         <plugin>
59             <groupId>org.springframework.boot</groupId>
60             <artifactId>spring-boot-maven-plugin</artifactId>
61         </plugin>
62     </plugins>
63 </build>
64

```

3) 修改配置文件

在配置文件中添加如下配置。

```

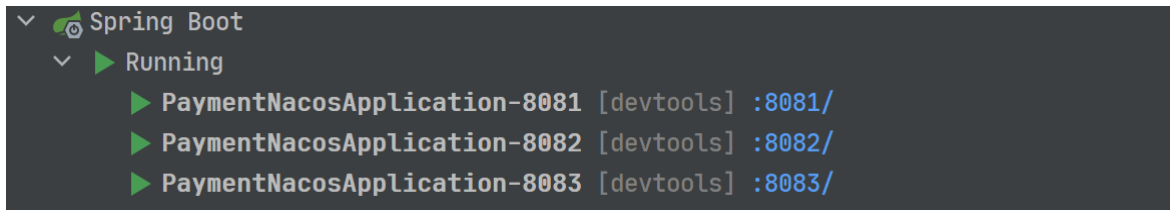
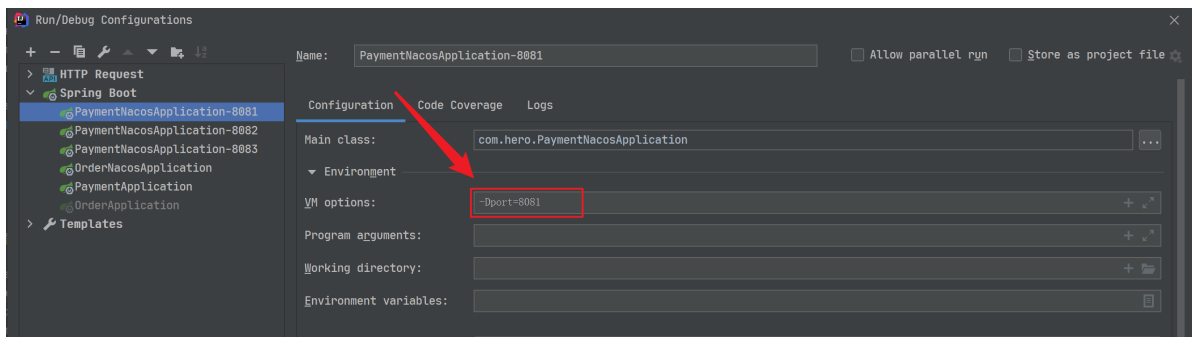
1  server:
2      # 表示，没有port参数，使用9001端口
3      port: ${port:8081}
4  spring:
5      cloud:
6          # 指定nacos注册中心的地址
7          nacos:
8              discovery:
9                  server-addr: 127.0.0.1:8848
10         # 指定微服务名称
11         application:
12             name: msc-payment
13     # 配置日志
14     logging:
15         pattern:
16             console: level-%level %msg%n
17         level:
18             root: info
19             com.hero: debug

```

4) 启动多个服务

启动两个服务实例端口号分别为8081、8082、8083，注册到nacos中

```
1 -Dport=8081
```



集群: DEFAULT

元数据过滤					
key		value		添加过滤	
IP	端口	临时实例	权重	健康状态	元数据
192.168.56.1	8081	true	1	true	preserved.register.source=SPRING_CLOUD
192.168.56.1	8082	true	1	true	preserved.register.source=SPRING_CLOUD
192.168.56.1	8083	true	1	true	preserved.register.source=SPRING_CLOUD

2.3.2 改造01-order微服务

1) 复制工程

复制 01-order工程，并重命名为 02-order-nacos。这个消费者是通过RestTemplate 进行消费的。

2) 修改 pom

与定义 provider 时的相同，consumer 的 pom 也做了如下几处的修改：

- 添加 spring cloud 依赖
- 添加 spring cloud alibaba 依赖
- 添加 nacos-discovery 依赖

```
1 <properties>
2   <java.version>1.8</java.version>
3   <spring-cloud.version>2020.0.1</spring-cloud.version>
4   <spring-cloud-alibaba.version>2021.1</spring-cloud-alibaba.version>
5 </properties>
6
7 <dependencies>
8   <!--nacos discovery依赖-->
9   <dependency>
10    <groupId>com.alibaba.cloud</groupId>
11    <artifactId>spring-cloud-starter-alibaba-nacos-
12    discovery</artifactId>
13    </dependency>
14    <dependency>
```

```
15     <groupId>org.springframework.cloud</groupId>
16     <artifactId>spring-cloud-starter-loadbalancer</artifactId>
17 </dependency>
18
19 <dependency>
20     <groupId>org.springframework.boot</groupId>
21     <artifactId>spring-boot-starter-web</artifactId>
22 </dependency>
23 <dependency>
24     <groupId>org.springframework.boot</groupId>
25     <artifactId>spring-boot-devtools</artifactId>
26     <scope>runtime</scope>
27     <optional>true</optional>
28 </dependency>
29 <dependency>
30     <groupId>org.springframework.boot</groupId>
31     <artifactId>spring-boot-starter-test</artifactId>
32     <scope>test</scope>
33 </dependency>
34 <dependency>
35     <groupId>com.hero</groupId>
36     <artifactId>spring-cloud-common</artifactId>
37     <version>1.0-SNAPSHOT</version>
38 </dependency>
39 </dependencies>
40
41 <dependencyManagement>
42     <dependencies>
43         <dependency>
44             <groupId>org.springframework.cloud</groupId>
45             <artifactId>spring-cloud-dependencies</artifactId>
46             <version>${spring-cloud.version}</version>
47             <type>pom</type>
48             <scope>import</scope>
49         </dependency>
50         <dependency>
51             <groupId>com.alibaba.cloud</groupId>
52             <artifactId>spring-cloud-alibaba-dependencies</artifactId>
53             <version>${spring-cloud-alibaba.version}</version>
54             <type>pom</type>
55             <scope>import</scope>
56         </dependency>
57     </dependencies>
58 </dependencyManagement>
59 <build>
60     <plugins>
61         <plugin>
62             <groupId>org.springframework.boot</groupId>
63             <artifactId>spring-boot-maven-plugin</artifactId>
64         </plugin>
65     </plugins>
66 </build>
67
```

3) 修改配置文件

在配置文件中添加如下内容：

```
1  server:
2      port: 8080
3  spring:
4      application:
5          name: msc-order
6      cloud:
7          nacos:
8              discovery:
9                  server-addr: 127.0.0.1:8848
10  ## 配置日志
11  logging:
12      pattern:
13          console: level-%level %msg%n
14      level:
15          root: info
16          com.hero: debug
```

4) 修改处理器类

原来 consumer 中采用的是直连方式访问 provider，现在要根据微服务名称来访问。

```
12  @RestController
13  @RequestMapping("/order")
14  public class OrderController {
15
16      @Autowired
17      private RestTemplate restTemplate;
18
19      //private static final String payment_service = "http://localhost:8081";
20      private static final String payment_service = "http://msc-payment";
21
22      @GetMapping("/payment/{id}")
23      public ResponseEntity<Payment> getPaymentById(@PathVariable("id") Integer id) {
24          //这么做存在什么问题?
25          //http://localhost:8081/payment/666
26          String url = String.format("%s/payment/%d", payment_service, id);
27          /** 问题01-消费者直接连接的提供者 ...*/
28
29          Payment payment = restTemplate.getForObject(url, Payment.class);
30          return ResponseEntity.ok(payment);
31      }
32  }
```

5) 配置负载均衡器

当使用微服务名称来访问 payment 时，其是通过负载均衡方式进行访问的。所以，需要添加如下注解。这里的负载均衡采用的是 spring cloud 自己开发的 spring cloud loadbalancer。

```

9  @SpringBootApplication
10 public class OrderNacosApplication {
11
12     public static void main(String[] args) { SpringApplication.run(OrderNa
13
14
15
16     /** Spring提供了一个RestTemplate模板工具类，对基于Http的客户端进行了封装，并且实
17
18
19
20
21     @Bean
22     @LoadBalanced//配置开启负载均衡器
23     public RestTemplate restTemplate() { return new RestTemplate(); }
24
25
26 }

```

2.4 将数据持久到外置 MySQL

默认情况下，Nacos 中的配置数据是被持久到内置的 MySQL 数据库中的，但使用内置数据库，存在很多问题。生产环境下，Nacos 一般会连接到外部的 MySQL。当然，目前 Nacos 仅支持 MySQL，并且要求是 5.6.5 及其以上版本。

2.4.1 查找 SQL 脚本文件

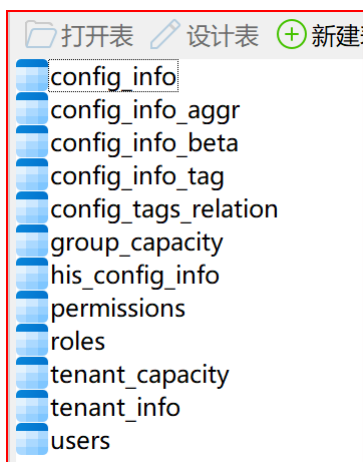
若要连接外置 MySQL，则外置 MySQL 中就要有相应的数据库 nacos_config 及表。这些表的创建语句 Nacos 官方已经给出了 SQL 的脚本文件，在 Nacos 解压目录的 config 子目录中 nacos-mysql.sql。

首先创建数据库 nacos_config

数据库名:	nacos_config
字符集:	utf8mb4 -- UTF-8 Unicode
排序规则:	utf8mb4_general_ci

2.4.2 运行脚本文件

脚本文件为 Nacos 解压目录的 config 子目录中 nacos-mysql.sql。运行该脚本文件，在 DBMS 中可以查看到创建的 DB 及表。



2.4.3 修改 Nacos 配置

打开 Nacos 安装目录下的 conf/application.properties 文件。

```
32 ### If use MySQL as datasource:
33 spring.datasource.platform=mysql
34
35 ### Count of DB:
36 db.num=1
37
38 ### Connect URL of DB:
39 db.url.0=jdbc:mysql://127.0.0.1:3306/nacos_config?character
40 db.user.0=root
41 db.password.0=root
```

2.4.4 重启服务：看到use external storage表示成功

```
2022-12-07 14:34:18,222 INFO Initializing ExecutorService 'taskScheduler'
2022-12-07 14:34:18,249 INFO Exposing 2 endpoint(s) beneath base path '/actuator'
2022-12-07 14:34:18,398 INFO Tomcat started on port(s): 8848 (http) with context path '/nacos'
2022-12-07 14:34:18,405 INFO Nacos started successfully in stand alone mode. use external storage
2022-12-07 14:34:18,906 INFO Initializing Servlet 'dispatcherServlet'
2022-12-07 14:34:18,920 INFO Completed initialization in 13 ms
```

2.5 Nacos 集群搭建

无论是 Nacos Discovery 还是 Nacos Config，单机版 Nacos 都存在单点问题。所以需要搭建高可用的 Nacos 集群。

搭建三台 Nacos

(1) 修改配置并复制目录

首先随意创建一个目录，用于存放三个 Nacos 服务器。然后再复制原来配置好的单机版的 Nacos 到这个目录，并重命名为 nacos8847。

打开 nacos8847/conf，重命名其中的 cluster.conf.example 为 cluster.conf。然后打开该文件，在其中写入三个 nacos 的 ip:port。

```
cluster.conf
14 # limitations under the License.
15 #
16
17 #it is ip
18 #example
19 10.20.5.11:8847
20 10.20.5.11:8848
21 10.20.5.11:8849
```

然后再打开 nacos8847/conf/application.properties 文件，修改端口号为 8847，配置外置MySQL。


```
cluster.conf application.properties
16
17 ***** Spring Boot Related Configurations *****#
18 ### Default web context path:
19 server.servlet.contextPath=/nacos
20 ### Default web server port:
21 server.port=8847
22
```

```
35 ### Count of DB:
36 db.num=1
37
38 ### Connect URL of DB:
39 db.url.0=jdbc:mysql://127.0.0.1:3306/nacos_config?characterE
40 db.user.0=root
41 db.password.0=root
```

然后再将 nacos8847 目录复制三份，分别命名为 nacos8848、nacos8849。并重新指定端口号分别为 8848 与 8849。

名称	
nacos-8847	2
nacos-8848	2
nacos-8849	2

(2) 启动集群

```
1 D:\nacos-cluster\nacos-8847\bin\startup.cmd -m cluster
2 D:\nacos-cluster\nacos-8848\bin\startup.cmd -m cluster
3 D:\nacos-cluster\nacos-8849\bin\startup.cmd -m cluster
```

```
D:\nacos-cluster\nacos-8849\conf>D:\nacos-cluster\nacos-8849\bin\startup.cmd -m cluster
"nacos is starting with cluster"

Nacos 1.4.1
Running in cluster mode, All function modules
Port: 8849
Pid: 10112
Console: http://192.168.56.1:8849/nacos/index.html
https://nacos.io

2022-12-07 14:53:57,234 INFO The server IP list of Nacos is [127.0.0.1:8847, 127.0.0.1:8848, 127.0.0.1:8849]
```

(3) 服务注册集群

```
1 server:
2   # 表示，没有port参数，使用9001端口
3   port: ${port:8082}
4 spring:
5   cloud:
6     # 指定nacos注册中心的地址
7     nacos:
8       discovery:
9         server-addr: 127.0.0.1:8847,127.0.0.1:8848,127.0.0.1:8849
10    # 指定微服务名称
11    application:
12      name: msc-payment
```

```
13 # 配置日志
14 logging:
15   pattern:
16     console: level-%level %msg%n
17   level:
18     root: info
19     com.hero: debug
```